Intelligent Incident Surveillance Response System

A Project Report

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science & Engineering

by

Ashna Joy

TVE22CS-P012

Computer Science & Engineering

GOVERNMENT ENGINEERING COLLEGE

Thiruvananthapuram

November 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Government Engineering College Thiruvananthapuram

2022 - 2026

CERTIFICATE

This is to certify that the report entitled **Intelligent Incident Surveillance Response System** submitted by **Ashna Joy** (TVE22CS-P012), to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science & Engineering is a bona fide record of the project work carried out by her under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Prof.George Thomas
(Project Guide & Coordinator)
Assistant Professor
Dept.of CSE

External Auditor

DECLARATION

I, Ashna Joy, member of Team 3, consisting of Akhil M Sasi, Justin Solomon, and Swathi GS, hereby declare that the project report entitled **Intelligent Incident Surveillance Response System**, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology of APJ Abdul Kalam Technological University, Kerala, is a bona fide record of work carried out by me under the supervision of Prof.George Thomas.

This submission represents my original work. Wherever the ideas or words of others have been included, they have been properly and accurately acknowledged through citation and reference.

I further declare that I have adhered to the principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any data, fact, or source in this submission. I understand that any violation of the above will invite disciplinary action by the institute and/or the University. This report has not previously formed the basis for the award of any degree, diploma, or similar title of any other university or institution.

Trivandrum

23-10-2025

Ashna Joy

ABSTRACT

This project implemented an intelligent vehicle surveillance system that records and uploads video only when a relevant event occurs. The system was designed to improve vehicular security and situational awareness by combining event detection, buffered video capture, cloud upload, and mobile access within a modular architecture. Instead of relying on continuous recording, the system preserves a rolling pre-event buffer and stores post-event footage so that each incident is captured with adequate context while reducing unnecessary storage usage.

The prototype was developed on a Linux-based platform using independent modules for event detection, event coordination, video recording, cloud upload, and mobile access. Trigger events such as honking, sudden braking, alarm activation, and suspicious person detection initiate recording. Video is encoded into MP4 format, uploaded to a server, and made available through an Android application that also receives event notifications. The implementation demonstrates the feasibility of integrating computer vision, multimedia processing, cloud services, and mobile connectivity into a practical surveillance workflow.

The completed system provides event-driven recording, remote access to incident footage, and real-time user notification. The report discusses the design decisions, implementation details, functional validation, and limitations of the prototype, and shows how the proposed approach offers a more efficient and responsive alternative to passive dash-camera systems.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, **Prof.George Thomas**, Assistant Professor , Department of Computer Science & Engineering, Government Engineering College, Thiruvananthapuram, for the valuable guidance, encouragement, and support provided throughout the course of this work. His suggestions and supervision were instrumental in the successful completion of this project.

I also extend my sincere thanks to **Prof.George Thomas** , for their support, cooperation, and constructive feedback during the progress of the project. I am grateful to **Dr.Jayasree P K**,Head of Part time Computer Science Department, for providing the facilities and academic environment required for this work.

Finally, I thank my family and friends for their constant encouragement and support throughout the completion of this project.

Ashna Joy

Contents

Abstract **i**

Acknowledgement **ii**

List of Figures **vii**

Abbreviations **viii**

1 Introduction **1**

1.1 Background to the Project 1

1.2 Project Objectives 2

2 Literature Review **4**

2.1 GSM-Based Alert System 4

2.2 Continuous Dashcam Recording 5

2.3 Live Surveillance Systems 6

2.4 Proprietary Surveillance 8

2.5 COSENS: Collaborative and opportunistic sensing of road events by vehicles' cameras 9

2.6 Cloud-assisted event detection in vehicular networks 10

2.7 Real-time data streaming in connected vehicles. Journal of Computer Networks and Communications 11

3 Problem Definition **12**

3.1 Problem Statement 12

3.2 Problem Description 13

4 Methodology **15**

4.1 Scrum Framework Overview 15

4.1.1 Scrum Team Roles 15
4.1.2 Scrum Artifacts 16
4.2 Team Composition and Responsibilities 16
4.3 Sprint Planning and Execution 17
4.3.1 Sprint 1 (03 Nov – 21 Nov 2025): System Integration & Database Architecture 17
4.3.2 Sprint 2 (01 Dec – 19 Dec 2025): Network Streaming & Event Logic Refinement 17
4.3.3 Sprint 3 (05 Jan – 23 Jan 2026): Integration & Testing 18
4.3.4 Sprint 4 (02 Feb – 20 Feb 2026): Collaborative Logic & Performance Testing 18
4.3.5 Sprint 5 (02 Mar – 20 Mar 2026): Testing, Documentation & Final Review 18
4.4 Sprint Summary 19
4.5 Advantages of Agile/Scrum for This Project 19

5 Requirements, Analysis and Design **20**
5.1 Requirements Gathering 20
5.2 System Design 21
5.2.1 Event Sources and Triggers 21
5.2.2 UNIX Platform and Modular Core 22
5.2.3 Cloud Backend 22

6 High-Level Design (HLD) **23**
6.1 Architecture Overview 23
6.2 Major Subsystems 23
6.3 Component Responsibilities 23
6.4 Data Flow Summary 24
6.5 Design Constraints 24
6.6 Design Rationale 24

7 UML Diagram **25**
7.1 Use Case Diagram (Textual) 25
7.2 Class Diagram (Conceptual) 26
7.3 Sequence Diagram (Event to Notification) 27

7.4 Deployment Diagram (Logical Nodes) 27

8 User Guide **28**

8.1 Purpose 28

8.2 Prerequisites 28

8.3 Project Files of Interest 28

8.4 Startup Procedure 29

8.5 Operational Workflow 29

8.6 Troubleshooting 29

8.7 Best Practices 29

9 Module Description **30**

9.1 Event App 30

9.2 Event Manager 30

9.3 Video Manager 31

9.4 Cloud Manager 31

9.5 Mobile Application 31

10 Implementation **32**

10.1 Event Detection (event app) 32

10.2 Event Manager (event manager) 32

10.3 Video Manager (video manager) 33

10.4 Cloud Manager (cloud manager) 33

10.5 Mobile App (Android) 33

10.6 Development Approach 33

10.7 Deployment 34

11 Result **35**

12 Conclusion **37**

References **38**

A Annex: Software Requirements Definition (SRD) **39**

A.1 Purpose 39

A.2 Scope 39

A.3 Stakeholders 40

A.4 Business and User Needs 40
A.5 Assumptions and Constraints 40
A.6 Success Criteria 40

B Annex: Software Requirements Specification (SRS) 41
B.1 Introduction 41
B.2 Functional Requirements 41
B.3 Non-Functional Requirements 42
B.4 External Interface Requirements 42
B.4.1 Software Interfaces 42
B.4.2 Communication Interfaces 43
B.5 Traceability Summary 43

C Annex: Unit Testing (UT) 44
C.1 Objective 44
C.2 Unit Test Scope 44
C.3 Test Environment 44
C.4 Representative Unit Test Cases 45
C.5 Pass Criteria 45
C.6 Risk-Based Notes 45

List of Figures

5.1 System Architecture 21

7.1 Use Case Diagram 25
7.2 Class Diagram 26
7.3 Sequence Diagram 27
7.4 Deployment Diagram 27

10.1 System Flow Diagram 34

11.1 Person Detection 36
11.2 Mobile App 36

Abbreviations

API	Application Programming Interface
FCM	Firebase Cloud Messaging
FIFO	First In, First Out
GSM	Global System for Mobile Communications
GPS	Global Positioning System
GStreamer	Open-source multimedia framework
HTTP	Hypertext Transfer Protocol
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
MP4	MPEG-4 Part 14 video container format
MySQL	Structured query database management system
OpenCV	Open Source Computer Vision Library
PHP	Hypertext Preprocessor
REST	Representational State Transfer
UI	User Interface
URL	Uniform Resource Locator
USB	Universal Serial Bus
Wi-Fi	Wireless Fidelity

Chapter 1

Introduction

This section provides the contextual foundation for the Smart Vehicle Surveillance System project. It begins by highlighting the limitations of conventional vehicle security solutions and outlines the real-world issues that motivated the development of an intelligent, event-triggered monitoring system. The project's primary objectives are enhancing vehicular situational awareness, optimized video recording based on meaningful triggers, and improving real-time user engagement and are discussed in relation to existing gaps. Finally, this section gives an overview of how the proposed system serves potential users such as private car owners, fleet managers, and rental services, and introduces the structure of the report, which includes detailed discussions on literature review, methodology, system design, implementation, and evaluation.

1.1 Background to the Project

With the rapid increase in vehicle ownership and urban congestion, vehicular security has become a growing concern for both individuals and organizations. While modern cars now come equipped with a range of built-in safety features such as GPS tracking, central locking systems, and basic alarm mechanisms, these solutions tend to be passive in nature. One of the most common additions for vehicle owners is the use of dash cameras, which continuously record video regardless of whether an incident is taking place. Although helpful in accident documentation or evidence collection, this continuous recording leads to excessive consumption of storage resources, often capturing long stretches of irrelevant footage. As a result, reviewing such footage post-event becomes time-consuming and inefficient for users.

More critically, existing systems fail to provide intelligent, context-aware monitoring that distinguishes between normal driving behaviour and security-related incidents such as

attempted theft, break-ins, vandalism, or suspicious proximity of individuals. There is also a significant lack of real-time user engagement, meaning that vehicle owners often remain unaware of potential threats until after damage has occurred.

This project addresses that gap by proposing a Smart Vehicle Surveillance System capable of detecting specific trigger events (such as sudden braking, honking, alarm activation, or unusual human movement near the car) and responding immediately. The system not only records footage based on relevant triggers but also notifies the vehicle owner in real time and provides access to recorded clips through a dedicated mobile application ("Security Viewer"). Although the project was not commissioned by a specific industry client, it was conceptualized with the aim of addressing a widespread problem affecting everyday vehicle owners, fleet operators, and car rental services. These user groups could significantly benefit from enhanced situational awareness, incident-based video evidence, and proactive security alerts that enable timely decisions or interventions. By leveraging existing in-vehicle cameras (such as surround view systems), the proposed solution also emphasizes cost-effectiveness and practical integration without requiring major modifications to the vehicle.

1.2 Project Objectives

The main objective of this project was to develop a modular, event-triggered video surveillance system that integrates real-time video recording, intelligent event detection, cloud storage, and mobile access. The following specific objectives were defined at the start of the project:

1.**Real-Time Event Detection**

To implement an event detection mechanism using both external simulation and computer vision techniques (such as detecting suspicious persons using OpenCV) that can trigger the recording process.

2.**Buffered Video Recording**

To design a system that captures not only post-event footage but also pre-event video using a memory-efficient circular queue, ensuring a complete 60-second context (30 seconds before and after the event).

3.**GStreamer-Based Video Encoding**

To use GStreamer for real-time video encoding and mixing, generating MP4 files that are timestamp-accurate and compatible with playback on standard media players and mobile apps.

4.**Modular Component Design**

Implement a modular architecture, where each subsystem (event app, event manager, video

manager, and cloud manager) runs as an independent process and communicates through Linux IPC mechanisms (named pipes).

5.**Cloud Upload System**

To develop a locally hosted cloud back-end using Apache, PHP, and MySQL,
capable of storing uploaded videos and their associated metadata,
accessible via RESTful APIs.

6.**Android Application Integration**

Build an Android mobile application that:

- Lists all recorded and uploaded videos,
- Plays selected recordings, and
- Receives real-time notifications whenever new footage is uploaded.

7.**End-to-End Workflow Integration**

To integrate all modules into a working pipeline from event detection to video recording, cloud upload, and mobile notification, providing a fully automated surveillance system.

Chapter 2

Literature Review

2.1 GSM-Based Alert System

The article by Patel et al. [1] addresses the persistent issue of vehicle theft, which continues despite advancements in automotive technology. Modern theft techniques such as relay attacks, CAN bus manipulation, and key fob cloning have made traditional anti-theft measures less effective. To counter this, the authors propose a multi-layered car security system that integrates authentication, tracking, and alert mechanisms. The system employs smart authentication methods like RFID, biometrics, or mobile apps to ensure only authorized access, while GPS and GSM modules enable real-time vehicle tracking in case of unauthorized movement. An engine immobilizer can remotely disable the ignition if suspicious activity is detected, and instant alerts are sent to the owner through SMS or mobile notifications containing details like time, location, and security status. Optional camera integration for visual monitoring is also suggested to aid in evidence collection. The system's strengths lie in its layered defence model, combining proactive prevention, reactive response, and remote monitoring, while its modular design allows easy adaptation to various vehicle types using common technologies like GSM, GPS, and microcontrollers.

However, challenges remain, including the need for secure communication channels to prevent data interception, the risk of false alerts due to sensor sensitivity, and concerns over power consumption from continuous module operation. The lack of experimental data also limits the evaluation of its real-world effectiveness. This study is closely related to the current research on Smart Vehicle Surveillance Systems, sharing features like event-triggered responses and owner notifications, but differing by focusing primarily on theft prevention rather than surveillance. In contrast, the ongoing research extends the concept by incorporating video

recording, cloud storage, and a mobile application for enhanced user interaction. Overall, Patel et al.'s work contributes meaningfully to vehicle security research by presenting a feasible and cost-effective approach, though further development is needed in encryption, data validation, and real-world testing to strengthen its practicality and reliability.

2.2 Continuous Dashcam Recording

The study by Rocky et al. [2] emphasizes the growing need for reliable accident detection systems that utilize dashcam videos, addressing the inherent difficulty of detecting real-world accidents due to their rarity and complexity. With the widespread use of affordable dashcams, even in non-autonomous vehicles, this research focuses on leveraging visual data to identify and predict collisions or near-miss incidents, ultimately enhancing road safety for both human-driven and autonomous systems. The authors classify existing detection approaches into supervised, self-supervised, and unsupervised methods. Supervised models rely on labelled accident data and commonly use architectures like CNNs, RNNs, and attention-based temporal models, while self-supervised and unsupervised methods detect anomalies through reconstruction errors or feature inconsistencies when labelled data is limited. The review also explores spatial-temporal attention mechanisms and anomaly detection frameworks, referencing models such as the Dynamic Spatial-Temporal Attention (DSTA) network for improved event recognition.

In evaluating datasets and metrics, Rocky et al. provide an extensive overview of benchmark collections, including both early datasets and newer large-scale ones, alongside evaluation parameters like accuracy, precision, recall, F1-score, and early detection time, an essential factor in proactive safety systems. However, they note a lack of diversity in existing datasets, emphasizing the need for samples across various weather conditions, lighting environments, and collision types. The strength of the review lies in its comprehensive scope, coherent classification of methods, and insightful discussion of critical challenges such as data scarcity, rare-event detection, and the complexities of early accident identification. The authors also offer practical recommendations to improve future research, such as the creation of diverse datasets and the standardization of evaluation protocols to enable fair comparison across models.

Despite these strengths, several challenges persist. The scarcity of labelled data limits the effectiveness of supervised models, while generalization across different driving scenarios remains problematic. Achieving timely yet accurate detection without excessive false alarms

continues to be a key issue, and the lack of consistent evaluation standards across studies hinders reliable benchmarking. The review holds strong relevance to the ongoing Smart Vehicle Surveillance project, as it reinforces core concepts like event-driven video capture and analysis, similar to our system's honk or braking-triggered recording mechanisms. It also highlights the potential of attention-based architectures for continuous dashcam data processing and stresses the importance of dataset diversity, which aligns with our plan to evaluate the system under multiple scenarios and edge cases.

In conclusion, Rocky et al.'s work provides a valuable synthesis of dashcam-based accident detection research, organizing the field with clarity while identifying crucial gaps and future directions. The authors recommend building larger, more diverse datasets, developing semi-supervised or self-supervised models to address limited data availability, and establishing standardized benchmarks for consistent performance evaluation. These insights can directly inform the Smart Vehicle Surveillance project by guiding the implementation of attention-based real-time detection, structured testing pipelines, and unsupervised methods for recognizing abnormal driving events that trigger automated recording and alert systems.

2.3 Live Surveillance Systems

Rehman et al. [3] address the growing need for real-time remote vehicle surveillance by proposing an IoT-based system that allows car owners to monitor live video streams through a mobile application whenever unauthorized access is detected. Unlike traditional alarm systems that only notify users after an intrusion has occurred, this work emphasizes instant awareness and visual feedback, enabling immediate action. The proposed system employs a Raspberry Pi 3 B+ as the central control unit, connected to six SW-420 vibration and motion sensors along with three strategically placed cameras to provide complete 360° coverage of the vehicle. When motion or vibration is detected on areas such as the doors, hood, or trunk, the GSM/GPRS module sends alerts to the vehicle owner while allowing two-way communication for remote commands. Upon receiving a notification, the owner can access live video streaming through the mobile app to visually confirm any suspicious activity. The setup also includes a dedicated 5V backup power supply that keeps the system functional even if the main vehicle battery is disconnected, and an optional cloud storage feature that allows users to save video clips for evidence or later review.

Technically, the system's design offers several advantages. The combination of Raspberry Pi and SW-420 sensors ensures prompt detection and alert generation, while the three-camera

configuration, positioned on the left and right pillars and the front mirror, provides comprehensive visual coverage of the vehicle's surroundings. The GSM/GPRS module facilitates instant communication for alerts and commands, and the auxiliary battery counteracts common sabotage attempts like disconnecting the main power. However, despite its innovation, the system also faces notable challenges. The GSM/GPRS network lacks advanced encryption, making it vulnerable to data interception. Latency issues in establishing live video streams could delay the owner's response time. Additionally, vibration sensors might trigger false alarms due to minor disturbances such as wind or nearby traffic, potentially reducing user confidence. Continuous operation of multiple sensors and modules may also drain power quickly, and large data transfers for live streaming and cloud storage can increase operational costs and bandwidth requirements.

This research is closely aligned with the objectives of the Smart Vehicle Surveillance project. Both systems utilize event-triggered video streaming, where recording or streaming begins in response to specific triggers such as suspicious movement or honking. The Security Viewer app developed in the current project builds upon Rehman et al.'s approach by adding functionalities like recorded playback, cloud integration, and a more interactive user interface. Moreover, while both systems rely on modular IoT components centered around the Raspberry Pi, the present work enhances this foundation with improved cloud deployment, mobile GUI design, and robust communication handling.

Overall, Rehman et al.'s Live View Car Surveillance System provides a strong architectural model for real-time vehicle monitoring, featuring complete coverage, live streaming, and user-driven control. Its modular and IoT-based design offers flexibility and scalability, serving as a valuable framework for further advancements. However, practical deployment requires improvements in areas such as data encryption, alert accuracy, latency reduction, and cost efficiency. The insights from this study significantly contribute to the development of the Smart Vehicle Surveillance project, guiding enhancements in real-time detection, user interface design, and the adoption of secure, reliable communication protocols.

2.4 Proprietary Surveillance

Tesla's Sentry Mode [4], introduced in 2023, represents a major advancement in proactive vehicle surveillance and security. Developed in response to increasing cases of vehicle break-ins and vandalism, it provides a smarter alternative to traditional alarm systems. As vehicle-related crimes surged, Tesla designed Sentry Mode to act as both a deterrent and an evidence-gathering mechanism. The system operates through a three-tiered state machine using external cameras and sensors. In Standby mode, it passively monitors the vehicle's surroundings. When minor disturbances are detected, such as someone leaning on the car, it enters Alert mode, displaying warnings on the touchscreen to discourage interference. In Alarm mode, triggered by significant threats like glass breaking, the system activates the car alarm, illuminates displays, blasts loud audio, and records footage beginning ten minutes prior to the incident. Simultaneously, alerts are sent to the owner's mobile app, and recorded videos are stored locally on a USB drive in a "Sentry" folder.

The strength of Sentry Mode lies in its intelligent design and comprehensive coverage. Its visible deterrence, displaying "Recording" messages on screens, helps discourage potential vandals. It leverages all of Tesla's Autopilot cameras to achieve complete 360° visual coverage, offering far greater monitoring capabilities than traditional front-facing dashcams. The system's contextual response mechanism ensures that reactions are appropriate to the level of threat, escalating only when necessary. Additionally, continuous pre- and post-event video recording provides valuable evidence for law enforcement and insurance claims. Real-world reports further validate its impact: incidents such as a vandal keying a Tesla in Oakland, vehicle damage cases in San Jose, and a failed carjacking attempt in New Zealand all demonstrate its practical deterrent value. In Sydney, Sentry footage helped identify a vandal, underscoring the system's contribution to community-level security and accountability.

However, Sentry Mode faces notable challenges. Continuous camera operation leads to considerable battery drain (up to 10–15 percent per day when parked). The system also demands manual activation after each parking session and lacks automated, location-based arming. Since recordings rely on physical USB drives, there is a risk of evidence loss if the device is stolen or damaged. Furthermore, by recording individuals in public without consent, the system raises privacy and legal concerns. Its functionality is also limited by cloud dependency; Tesla's app lacks native live-streaming or remote playback features without third-party integration.

Despite these limitations, Sentry Mode aligns closely with the objectives of the Smart

Vehicle Surveillance System. Both rely on event-triggered recording in response to motion, vibration, or intrusion. Tesla's use of existing camera hardware for multi-angle video capture parallels the surround-view setup proposed in the current project. Similarly, owner notifications through Tesla's app resemble the alert functionality in the Security Viewer app, though the latter expands capabilities with cloud uploads, remote playback, and real-time control. The Smart Vehicle Surveillance System also addresses Sentry Mode's shortcomings by incorporating encrypted communication, cloud redundancy, live streaming, battery optimization, and automated activation based on location or context.

Overall, Tesla's Sentry Mode: Guarding Your Tesla stands as a landmark innovation in commercial vehicle surveillance, successfully deterring crimes and recording crucial evidence. Its multi-layered detection strategy and user-centered feedback establish a strong foundation for next-generation security systems. Building upon this framework, the Smart Vehicle Surveillance project takes the concept further by adding enhanced automation, real-time interactivity, and secure cloud-based functionality, representing the natural evolution of intelligent vehicular surveillance technology.

2.5 COSENS: Collaborative and opportunistic sensing of road events by vehicles' cameras

The study "COSENS: Collaborative and Opportunistic Sensing of Road Events by Vehicles' Cameras" [5] by Arev, Park, Sheikh et al. focuses on enabling opportunistic and collaborative sensing of road events such as accidents and hazards using cameras mounted on ordinary vehicles, with real-time sharing among participants. In this approach, vehicles opportunistically capture video footage when specific events occur. The frame metadata, including GPS and time information, is shared and aggregated. The collected video snippets are collaboratively edited and highlighted across multiple sources, drawing inspiration from earlier multi-camera coordination frameworks such as Movi and Crowdtracking.

The study demonstrated the feasibility of using multiple non-specialized vehicle cameras to achieve collective incident awareness. It also showed efficient low-latency sharing of video data among participants, maintaining a synchronization window of approximately 200 milliseconds. Furthermore, the system enabled the generation of comprehensive multi-angle incident summaries using data from consumer-grade devices. However, certain gaps were identified in the system. It relies heavily on participants voluntarily opting into video sharing,

and there are challenges related to privacy control and access authorization. Additionally, synchronization accuracy depends on GPS and time precision, and there is no enforced mechanism for propagating incident alerts to nearby drivers or providing a continuous real-time video stream.

2.6 Cloud-assisted event detection in vehicular networks

Park and Lee [6] explore the integration of cloud computing with vehicular networks to improve real-time event detection and traffic management. In modern intelligent transportation systems, vehicles generate massive amounts of data through sensors, cameras, and communication modules, but processing all this information on individual vehicles is limited by computational power and storage capacity. To address these limitations, the authors propose a cloud-assisted framework where data collected by vehicles is transmitted to a centralized cloud platform. This enables large-scale analysis using advanced algorithms to detect traffic incidents, road hazards, abnormal driving behavior, and other events that may affect road safety. The paper emphasizes that offloading computation to the cloud not only reduces the processing burden on vehicles but also allows for faster and more accurate detection by leveraging the cloud's superior computational resources. The framework supports real-time communication between vehicles and the cloud, which is critical for timely alerts and preventive measures. Through simulations and experimental evaluation, the study demonstrates that the cloud-assisted approach significantly improves detection accuracy, reduces response times, and supports scalability, allowing the system to handle data from a large number of vehicles simultaneously. Furthermore, the authors discuss the potential benefits of such a system for traffic management authorities, including improved situational awareness, better traffic flow control, and enhanced safety for drivers and pedestrians. They also highlight the challenges associated with this approach, such as ensuring secure and reliable data transmission, managing latency in communication, and maintaining privacy of vehicular data. Overall, the paper provides a comprehensive analysis of how cloud computing can augment vehicular networks, showing that cloud-assisted event detection is a promising solution for enhancing road safety, optimizing traffic management, and enabling more intelligent and connected transportation systems.

2.7 Real-time data streaming in connected vehicles. Journal of Computer Networks and Communications

Ahmad and Khan [7] investigate real-time data streaming in connected vehicles, focusing on how continuous data exchange between vehicles and infrastructure can enhance traffic management and road safety. The study emphasizes that connected vehicles generate a vast amount of dynamic data from sensors, GPS modules, and onboard diagnostic systems, which must be transmitted, processed, and analyzed in real time to support applications such as collision avoidance, route optimization, and emergency response. The authors propose a framework for efficient data streaming that minimizes latency and ensures reliable communication among vehicles and between vehicles and cloud servers. They highlight the use of advanced networking protocols and data handling techniques to manage high-frequency data flows while maintaining scalability and robustness. Through simulations and experimental evaluations, the paper demonstrates that the proposed framework significantly reduces data transmission delays and improves the accuracy of real-time traffic information compared to traditional methods. Furthermore, Ahmad and Khan discuss the challenges associated with connected vehicle networks, including network congestion, data security, and privacy concerns, and suggest strategies to mitigate these issues. Overall, the study underscores the critical role of real-time data streaming in enabling intelligent transportation systems, allowing vehicles to respond dynamically to changing traffic conditions, enhancing road safety, and supporting the development of future autonomous driving technologies.

Chapter 3

Problem Definition

3.1 Problem Statement

With the rapid growth of vehicle ownership and urban traffic, ensuring vehicle security has become an increasingly critical concern for individuals and organizations alike. Conventional vehicle security systems, such as alarms, GPS trackers, and dash cameras, are widely used to protect vehicles from theft, vandalism, or accidents. While these systems provide a basic level of security, they often operate passively and lack the intelligence necessary to differentiate between normal vehicle activity and critical security events.

Dash cameras, for instance, continuously record video regardless of whether a noteworthy incident occurs. This continuous recording approach consumes significant storage space and often captures long periods of irrelevant footage. Reviewing such recordings to identify important events is time-consuming and inefficient, making it difficult for car owners to respond promptly to potential threats. Furthermore, existing systems rarely offer real-time alerts or remote access to footage, meaning that car owners are often unaware of a security event until it is too late to intervene or take preventive measures.

Moreover, current security solutions lack contextual understanding. Simple alarm notifications or GPS alerts cannot provide detailed situational awareness, such as identifying suspicious persons near the vehicle, detecting unusual vehicle movements, or capturing the moments immediately before and after an incident. This limitation restricts the ability of vehicle owners, fleet managers, and rental services to assess threats accurately and respond effectively.

The core problem, therefore, is the absence of a smart, event-driven vehicle surveillance system that can:

1. Detect security-relevant events intelligently using existing sensors and cameras.
2. Record video footage with pre- and post-event context to provide a complete understanding of incidents.
3. Notify vehicle owners in real time and allow remote access to recorded footage.
4. Efficiently use existing vehicle cameras to minimize cost and ensure easy integration.

By addressing these gaps, a smart vehicle surveillance system can significantly enhance vehicular security, situational awareness, and response effectiveness for car owners and other stakeholders.

3.2 Problem Description

The increasing number of vehicle-related incidents, including theft, vandalism, accidents, and unauthorized access, highlights the inadequacy of conventional vehicle security systems. Traditional solutions typically rely on reactive mechanisms, such as alarms or GPS tracking, which only provide basic information about an event after it has occurred. While dash cameras have become popular as a method of documenting incidents, their continuous recording strategy is inefficient, storing hours of irrelevant video and providing limited situational context.

A major challenge in existing systems is their inability to distinguish between ordinary events and security-critical events. For example, a vehicle might honk, brake suddenly, or encounter unusual human movement near its vicinity, but current surveillance systems cannot prioritize these events for immediate recording or alerting. Consequently, car owners may miss crucial moments that are essential for understanding an incident, verifying insurance claims, or aiding law enforcement investigations.

In addition to insufficient intelligence, most vehicle security systems lack integration with modern cloud-based platforms and mobile technologies. Remote access to video recordings is either unavailable or delayed, which reduces the effectiveness of the surveillance system. Without real-time notifications, vehicle owners cannot react promptly, leaving their cars vulnerable during critical incidents.

There is also a practical consideration: vehicle owners prefer solutions that leverage existing hardware, such as surround-view cameras or built-in sensors, to minimize installation costs and avoid significant modifications to the car. A cost-effective and modular system that can integrate with current vehicle infrastructure is therefore highly desirable.

The problem can be summarized as follows:

- Inefficient video recording: Continuous recording wastes storage and requires excessive manual review.
- Lack of event-based intelligence: Existing systems cannot prioritize or detect critical security events effectively.
- Absence of real-time engagement: Delayed or no notifications hinder timely response to threats.
- Limited integration with cloud/mobile: Owners lack remote access to footage, reducing the system's effectiveness.
- High deployment cost for new hardware: Many solutions require additional cameras or sensors, increasing cost and complexity.

To address these challenges, the proposed Smart Vehicle Surveillance System aims to provide an intelligent, event-driven monitoring solution. The system will detect security-relevant events, record pre- and post-event footage, notify the vehicle owner in real time, and store video securely in the cloud. By utilizing existing in-vehicle cameras and employing a modular architecture, the system ensures cost-effectiveness, scalability, and practical usability for car owners, fleet operators, and rental services.

The proposed approach bridges the gap between passive recording and active security, enhancing situational awareness and empowering users to take immediate action when an event occurs. It not only addresses the limitations of conventional systems but also provides a proactive solution capable of supporting insurance verification, law enforcement investigations, and overall vehicle safety.

Chapter 4

Methodology

The Intelligent Incident Surveillance Response System was developed using the **Agile methodology**, specifically following the **Scrum framework**. Agile was chosen because the project involved evolving technical requirements, multi-module integration across different team members, and the need for iterative validation of hardware–software interactions. The Scrum framework provided a structured cadence of short, focused sprints that allowed the team to deliver incremental working features, adapt to significant technical decisions mid-project (such as migrating the backend from C++ to Python), and maintain continuous alignment across all modules.

4.1 Scrum Framework Overview

Scrum organises work into time-boxed iterations called **sprints**. For this project each sprint was fixed at **three weeks**. At the start of each sprint, the team selects a set of backlog items to deliver. During the sprint, daily progress is tracked and blockers are addressed. At the end of each sprint, a working increment is demonstrated and a sprint review and retrospective are held to identify improvements for the next iteration.

4.1.1 Scrum Team Roles

- **Product Owner:** Collectively held by the team, responsible for maintaining the product backlog and prioritising features based on project objectives.
- **Scrum Master:** Responsibility rotated across team members to facilitate sprint planning, stand-ups, and retrospectives.

- **Development Team:** All four members contributed across backend, frontend, and integration domains.

4.1.2 Scrum Artifacts

- **Product Backlog:** A prioritised list of all features and requirements including event detection, circular buffer recording, cloud upload, notifications, and mobile playback.
- **Sprint Backlog:** Tasks committed to in each sprint, broken down from product backlog items.
- **Increment:** A working, tested portion of the system delivered at the end of every sprint.

4.2 Team Composition and Responsibilities

The development team consisted of four members, each holding primary ownership of a domain while collaborating on integration.

Team Member	Primary Role	Key Modules	Contributions
Akhil M Sasi	Core Systems & Infotainment	Android Infotainment App, Video Service	Developed the Jetpack Compose User App and Infotainment app, integrated storage manager and remote streaming pipeline, and added manual recording triggers.
Ashna Joy	User Experience & Mobile	Android User App, Video Gallery	Developed the User App UI and navigation flow; built the Python Surveillance Service including 30-second circular queue logic and local video encoding.
Swathi G S	Event & Analytics	MediaPipe Detection, Event Logic	Researched MediaPipe and built face detection scripts; developed the event manager for handling recording event types.
Justin Solomon	Integration & Data	Inter-process Communication (IPC)	Developed inter-process communication and the notification service across all modules.

4.3 Sprint Planning and Execution

The project was executed over **five 3-week sprints** spanning November 2025 to March 2026, each with a defined goal, planned activities, and a measurable status. A brief inter-sprint review period separated consecutive sprints to allow demonstration and backlog refinement.

4.3.1 Sprint 1 (03 Nov – 21 Nov 2025): System Integration & Database Architecture

Sprint Goal: Establish the core integration path between modules and set up the data persistence layer.

Key Activities:

- Integrated the video player into the Infotainment application.
- Optimised the mobile application gallery for performance.
- Calibrated detection algorithm thresholds for acceptable accuracy.
- Designed and deployed the MySQL database schema for incident metadata.

Status: *Completed*

4.3.2 Sprint 2 (01 Dec – 19 Dec 2025): Network Streaming & Event Logic Refinement

Sprint Goal: Enable real-time video streaming across the network and refine the backend event handling pipeline.

Key Activities:

- Added manual recording triggers to the user interface.
- Integrated local network streaming for MP4 incident files.
- Transitioned the backend from C++ to Python for improved MediaPipe and Firebase Admin SDK integration.

Status: *Completed*

4.3.3 Sprint 3 (05 Jan – 23 Jan 2026): Integration & Testing

Sprint Goal: Complete end-to-end integration of all independently developed modules and begin functional validation.

Key Activities:

- Finalised the signal handoff between the Surveillance Service and the Video Manager.
- Integrated all independent modules into a unified pipeline.
- Conducted functional testing and resolved integration-level bugs.

Status: *Completed*

4.3.4 Sprint 4 (02 Feb – 20 Feb 2026): Collaborative Logic & Performance Testing

Sprint Goal: Implement collaborative vehicle awareness features and validate system performance under challenging conditions.

Key Activities:

- Implemented "Nearby Cars" detection logic based on GPS location data.
- Automated the sharing of incident triggers to simulated nearby vehicles.
- Conducted performance validation in low-light and high-motion environments.

Status: *Completed*

4.3.5 Sprint 5 (02 Mar – 20 Mar 2026): Testing, Documentation & Final Review

Sprint Goal: Finalise the system, complete project documentation, and prepare for submission and evaluation.

Key Activities:

- Finalised end-to-end system testing and polished user interfaces.
- Fixed remaining bugs and conducted regression testing.
- Completed the project report and prepared final presentation materials.

Status: *Completed*

4.4 Sprint Summary

Sprint	Duration (3 weeks)	Focus Area	Status
Sprint 1	03 Nov – 21 Nov 2025	System Integration & Database Architecture	Completed
Sprint 2	01 Dec – 19 Dec 2025	Network Streaming & Event Logic Refinement	Completed
Sprint 3	05 Jan – 23 Jan 2026	Integration & Testing	Completed
Sprint 4	02 Feb – 20 Feb 2026	Collaborative Logic & Performance Testing	Completed
Sprint 5	02 Mar – 20 Mar 2026	Testing, Documentation & Final Review	Completed

4.5 Advantages of Agile/Scrum for This Project

- **Parallel Development:** Each team member could develop and test their module independently within a sprint, with integration at sprint boundaries.
- **Adaptability:** The mid-project migration from a C++ backend to Python was handled without disruption, which would have required a full reset under a rigid sequential model.
- **Incremental Delivery:** Each sprint delivered a testable increment, making it easier to validate system behaviour at every stage.
- **Continuous Feedback:** Sprint retrospectives allowed the team to continuously refine both technical and process-level decisions.

Chapter 5

Requirements, Analysis and Design

5.1 Requirements Gathering

The system must be able to record video footage in MP4 format from the connected cameras whenever a triggering event is detected.

- It should save a 30-second video segment prior to the event and an additional 30 seconds after the event has occurred.
- An event manager module must continuously monitor the system for various predefined event triggers.
- The system must be capable of sending real-time notifications to a mobile application, informing the user of an event, and providing access to the recorded video.
- The system should be able to detect the presence of a person as a valid trigger event using a person detection algorithm.
- The system must also detect the presence of a car as a trigger condition.
- The system should recognize and handle second-level trigger events, and initiate recording and notification processes accordingly.
- Recorded videos must be uploaded to a cloud server to enable remote viewing by the user.
- The system should allow the user to view the uploaded video on a mobile application interface immediately after the event is triggered.
- Record video around the detected event.

- Store video securely.
- Notify users via mobile

5.2 System Design

The design of the system was centred around building a modular, event-driven video recording and surveillance platform, capable of intelligently responding to real-world stimuli, recording relevant footage, and delivering notifications to a mobile application via a cloud backend. The overall system was implemented on a UNIX-based platform, with clear separation of concerns across multiple modules communicating through inter-process communication (IPC) using named pipes.

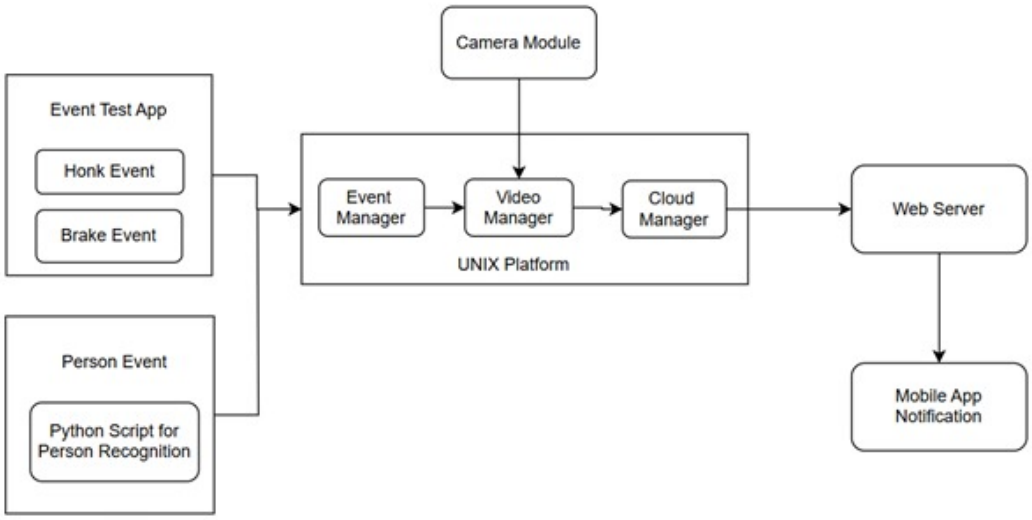

Figure 5.1: System Architecture

5.2.1 Event Sources and Triggers

There are two primary sources of events in the system:

- Event Test App:A simulation tool developed for triggering artificial events such as honk and brake events. This was essential during development and testing, allowing deterministic input and verification of system behaviour.
- Person Event Detection:Implemented as a Python script using OpenCV, this module performs real-time person detection using live video feed or frames captured from the

camera. Upon detecting a suspicious presence, it generates a person event and sends it to the Event Manager.

These inputs represent external triggers that cause the system to begin recording and initiate further downstream operations.

5.2.2 UNIX Platform and Modular Core

The core components of the system are executed on a UNIX platform. These include the event manager, video manager, and cloud manager modules.

- Event Manager: This is the central controller that receives input from both the event test app and the person detection script. It acts as the gateway to initiate the surveillance sequence. Upon receiving an event signal, it sends a control message to the video manager to begin processing and recording.

5.2.3 Cloud Backend

The Web Server is implemented using a local Apache installation, with PHP used to create REST APIs. These APIs handle:

- File reception and storage,
- Metadata insertion into a MySQL database,
- Notification triggers for newly uploaded files.

The web server acts as the central repository for all video files and associated event data.
Mobile App and Notification System:
Once the file is uploaded, the backend triggers a notification to the Mobile App. The app is designed to:

- Receive push notifications when a new event is uploaded,
- Fetch the list of available videos using the API,
- Allow users to play selected videos directly from the server.

This provides real-time access to surveillance events from a remote device, completing the end-to-end system flow from detection to user alert.

Chapter 6

High-Level Design (HLD)

6.1 Architecture Overview

The system follows a modular event-driven architecture. Each subsystem is isolated by responsibility and communicates through defined interfaces to reduce coupling and improve maintainability.

6.2 Major Subsystems

- Event Source Layer: generates events from simulation and vision detection.
- Orchestration Layer: validates events and triggers recording workflow.
- Recording Layer: builds incident clips from buffered and live frames.
- Cloud Layer: uploads clips and stores incident metadata.
- Client Layer: displays incidents and receives notifications.

6.3 Component Responsibilities

Component	**Primary File/Module**	**Responsibility**
Event Producer	data_monitor.py / debug_update.py	Emit controlled trigger events for test and validation flows.

Event Controller	EventManager.py	Receive and coordinate trigger handling across downstream modules.
Recording Service	recording_service.py	Manage incident clip creation and recording state transitions.
Cloud/Firebase Adapter	firebase_manager.py	Push notifications and cloud-facing update operations.
Operator UI	main_gui.py	Provide local control and status visibility for operators.
Utility Layer	utils.py	Shared helper functions used across modules.

6.4 Data Flow Summary

1. Event source emits trigger.
2. Event controller validates trigger and initiates recording action.
3. Recording service assembles incident clip with contextual buffer.
4. Cloud adapter uploads clip metadata and notifies clients.
5. Client views updated incident list and plays selected clips.

6.5 Design Constraints

- Timely event processing with low overhead on local hardware.
- Predictable inter-module interfaces for easier debugging.
- Compatibility with local backend deployment stack.

6.6 Design Rationale

This architecture prioritizes clarity and fault isolation. If one service degrades, logs and boundaries make diagnosis straightforward while preserving the ability to upgrade modules incrementally.

Chapter 7

UML Diagram

7.1 Use Case Diagram (Textual)

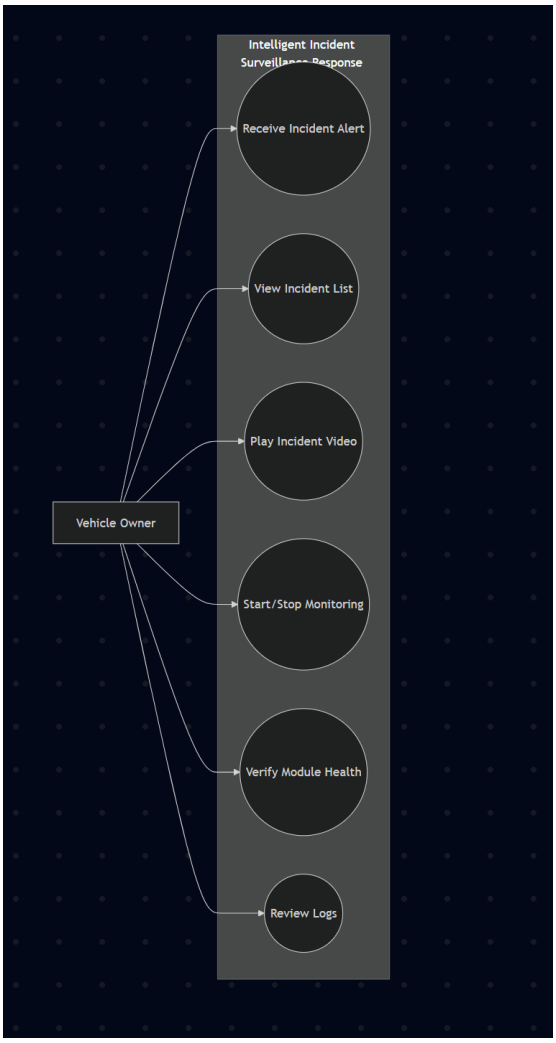

Figure 7.1: Use Case Diagram

7.2 Class Diagram (Conceptual)

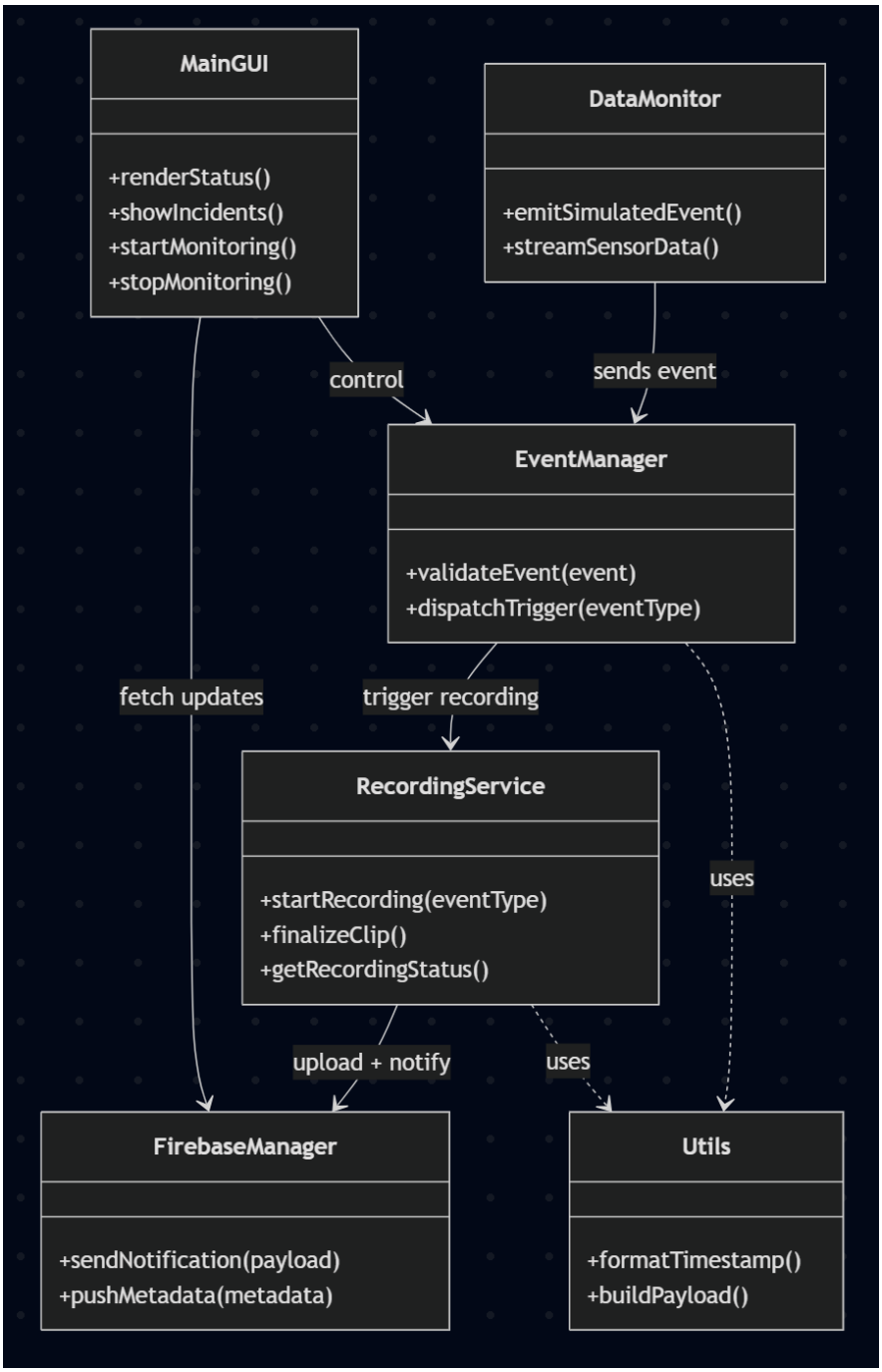

Figure 7.2: Class Diagram

7.3 Sequence Diagram (Event to Notification)

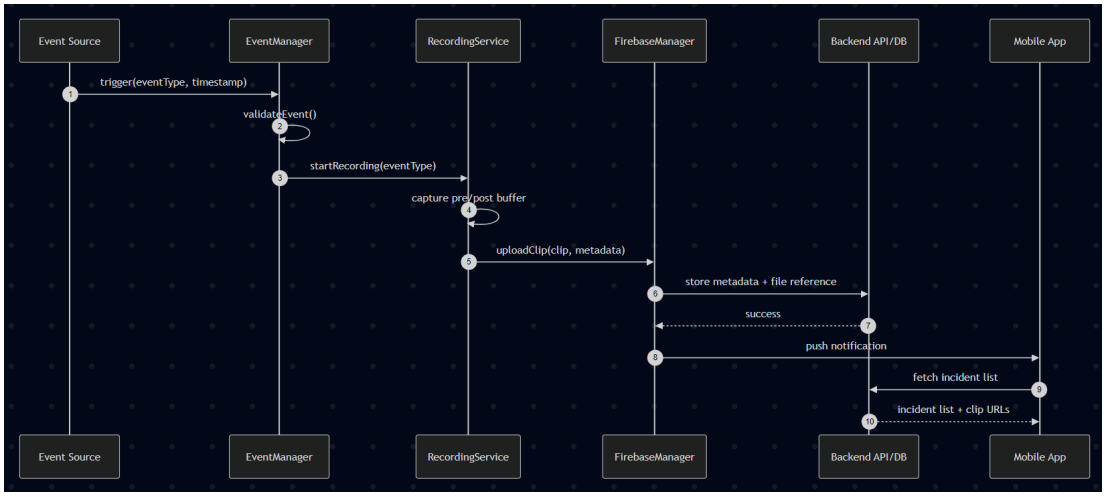

Figure 7.3: Sequence Diagram

7.4 Deployment Diagram (Logical Nodes)

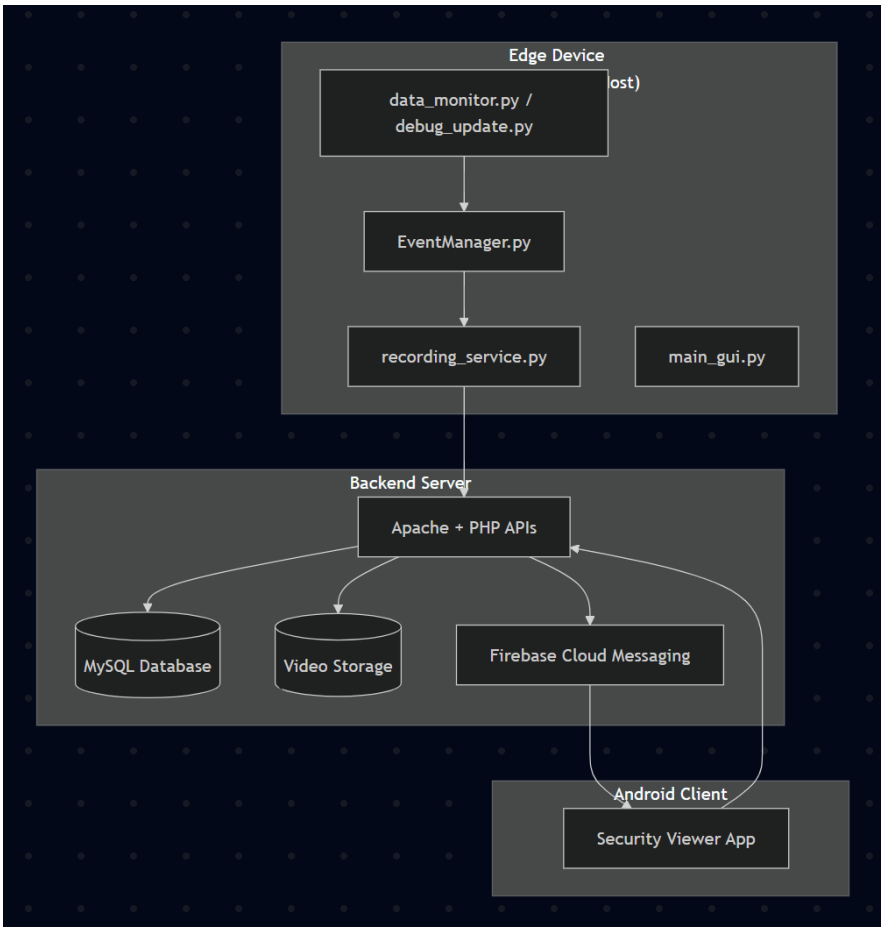

Figure 7.4: Deployment Diagram

Chapter 8

User Guide

8.1 Purpose

This guide explains how to run the Intelligent Incident Surveillance Response System in a local development setup and how to verify end-to-end behavior.

8.2 Prerequisites

- Python environment with project dependencies.
- Backend services configured (Apache, PHP, MySQL).
- Firebase credentials configured for notification features.
- Camera/video input source available for event simulation or detection.

8.3 Project Files of Interest

- main_gui.py: local control interface.
- EventManager.py: event coordination logic.
- recording_service.py: clip generation logic.
- firebase_manager.py: cloud/notification integration.
- data_monitor.py, debug_update.py, debug_firebase_test.py: testing and debug helpers.

8.4 Startup Procedure

1. Start backend services and verify database connectivity.
2. Confirm configuration files and credentials are present.
3. Launch monitoring and orchestration modules.
4. Launch the local GUI if interactive operation is required.
5. Trigger a test event and verify clip creation and upload.

8.5 Operational Workflow

1. System waits in monitoring state.
2. Trigger event occurs (simulated or vision-based).
3. Recording module builds incident clip with context.
4. Clip metadata is stored and notification is sent.
5. User opens client app and reviews incident list/playback.

8.6 Troubleshooting

- No notification received: check Firebase key path and backend notification logs.
- Clip missing: verify recording service status and available disk space.
- Upload failure: verify API endpoint reachability and backend permissions.
- Empty incident list: confirm metadata insertion in database tables.

8.7 Best Practices

- Keep system clock synchronized across modules.
- Use controlled event simulations before live camera testing.
- Archive old clips periodically to maintain storage performance.
- Review logs after each test cycle to catch intermittent issues.

Chapter 9

Module Description

The system was divided into five functional modules so that acquisition, decision-making, storage, and user interaction could be developed and tested independently. The modular design also simplified debugging and made it possible to replace or improve individual components without affecting the entire system.

9.1 Event App

The event app is responsible for generating trigger events for the surveillance pipeline. It supports both simulated events, such as honking or sudden braking, and computer-vision-based detection of suspicious human presence near the vehicle. The module was implemented in Python and used OpenCV for frame capture and preprocessing, while MediaPipe was used to strengthen human detection in live video streams. Whenever a valid event is detected, the module forwards an event message to the event manager through a named pipe.

9.2 Event Manager

The event manager acts as the central coordinator of the system. It receives event messages from the event app, validates the incoming trigger, and instructs the video manager to preserve the corresponding surveillance clip. This module was implemented in C++ to provide deterministic control and efficient inter-process communication. By separating event coordination from event generation, the system maintains clear control flow and supports future expansion to additional trigger sources.

9.3 Video Manager

The video manager performs the core recording task. It continuously maintains a circular buffer containing the most recent 30 seconds of video so that important context before a trigger event is not lost. Once an event is received, the module stores the buffered pre-event segment together with 30 seconds of post-event footage, producing a 60-second incident clip. GStreamer was used for encoding, muxing, and writing the final MP4 file, ensuring playback compatibility and efficient handling of the incoming video stream.

9.4 Cloud Manager

The cloud manager transfers the recorded video clip to the backend server and stores the associated metadata. The backend was implemented using Apache, PHP, and MySQL. On receiving a completed clip from the video manager, the cloud manager uploads the file through HTTP-based APIs and records relevant information such as file name, timestamp, and event type. This module enables remote access to the captured footage and serves as the bridge between the embedded platform and the mobile application.

9.5 Mobile Application

The Android application provides the user-facing interface of the system. It displays the list of uploaded recordings, supports playback of selected incident videos, and receives notifications whenever a new event is uploaded. This allows the vehicle owner to remain informed even when away from the vehicle. The mobile application therefore completes the end-to-end workflow by converting backend updates into immediate and usable information for the user.

Chapter 10

Implementation

The Smart Vehicle Surveillance System was developed using multiple programming languages, tools, and platforms. Its modular design allowed parallel development, isolated testing, and focused optimization of individual modules, reducing complexity and facilitating debugging.

The system was implemented on a UNIX-based Linux environment, which supports multi-process operations, inter-process communication, and real-time data processing. Modules communicated through UNIX named pipes (FIFOs), enabling efficient one-way streams between producer-consumer components.

10.1 Event Detection (event app)

- Implemented in Python.
- Simulated events (e.g., braking and honking) and real-world detection of suspicious individuals using OpenCV and MediaPipe.
- Human detection leveraged OpenCV for frame processing and MediaPipe models to identify persons within the camera frame.
- Detected events were sent to the Event Manager through a named pipe.

10.2 Event Manager (event manager)

- Developed in C++.
- Served as the central controller, interpreting events from the event app.
- Triggered the Video Manager to start recording when an event was confirmed.

10.3 Video Manager (video manager)

- Implemented in C++ using the GStreamer multimedia framework.
- Maintained a 30 second circular buffer of pre event frames.
- Upon trigger, combined pre event and 30 second post event video into a single MP4 file.
- GStreamer handled encoding, muxing, and buffering, ensuring reliable timing and video quality.

10.4 Cloud Manager (cloud manager)

- Managed the upload of MP4 files to a locally hosted Apache server.
- PHP scripts handled file reception via HTTP POST and stored metadata in a MySQL database.
- Used libcurl for reliable uploads and implemented retry mechanisms for network failures.

10.5 Mobile App (Android)

- Developed in Java for Android devices.
- Connected to the backend to fetch and display uploaded videos.
- Supported video playback using the native Android media player.
- Received real-time push notifications via Firebase Cloud Messaging (FCM) whenever a new video was uploaded.

10.6 Development Approach

- Early prototypes of each module were developed to test feasibility, library behavior, and system responsiveness.
- Iterative refinements were performed within modules while following the Agile/Scrum approach, such as optimizing the circular queue for memory usage and frame timing.
- Functional and integration testing ensured all modules worked together reliably.

10.7 Deployment

- The system is fully implemented, integrated, and tested in a local environment, including the Apache server, database, and connected cameras.
- The Android app has been tested on physical devices for playback and notifications.
- Deployment scripts automate service startup, network configuration, and headless operation on embedded UNIX platforms.

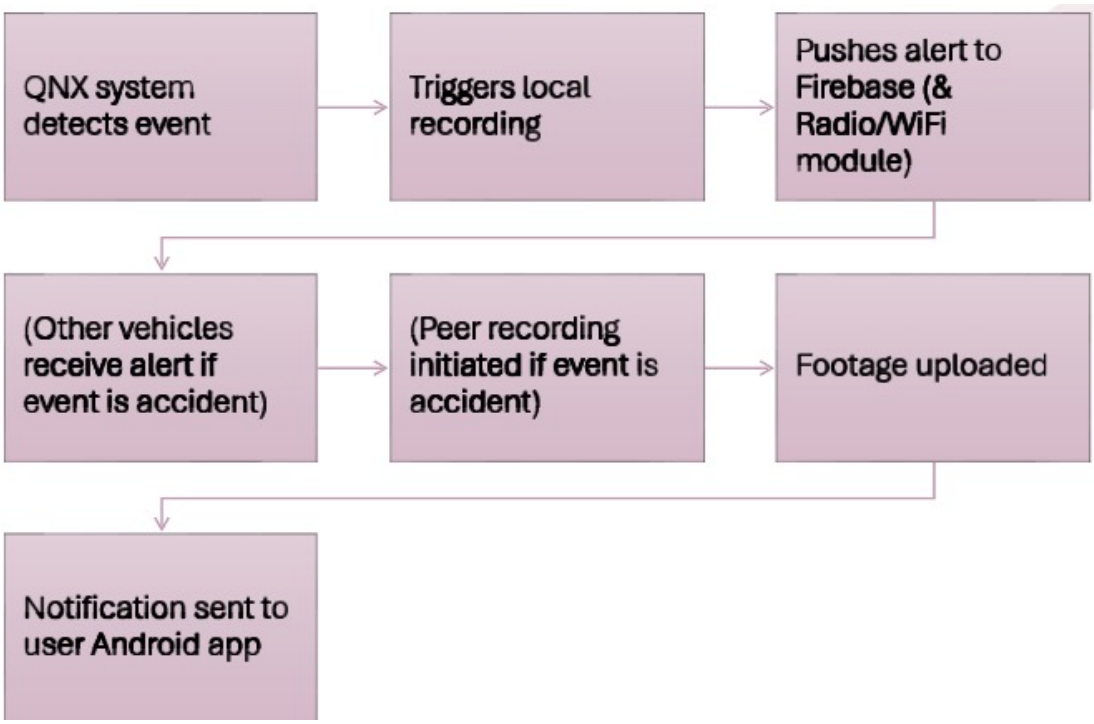

Figure 10.1: System Flow Diagram

Chapter 11

Result

The integrated prototype was functionally validated in a controlled local environment using simulated trigger events and computer-vision-based person detection. The evaluation focused on confirming the end-to-end behaviour of the system rather than establishing benchmark-level quantitative accuracy. The following outcomes were consistently observed during testing:

- Trigger events generated by the event app were correctly received by the event manager.
- The video manager preserved buffered pre-event footage and produced a complete incident clip with post-event context.
- Recorded clips were uploaded successfully to the local server along with the associated metadata.
- The Android application was able to list uploaded videos, play the selected recording, and receive event notifications after upload.

These results show that the proposed architecture is suitable for event-driven surveillance and remote incident review. The prototype demonstrated that selective recording can capture relevant evidence while reducing the storage overhead associated with continuous dash-camera operation. The screenshots presented in this chapter illustrate the person-detection stage and the mobile application interface used for remote access to recorded clips.

Although the functional objectives were achieved, formal quantitative evaluation remains a scope for future work. Metrics such as event-detection precision, false-positive rate, recording latency, cloud upload delay, and notification latency should be measured systematically in a larger test campaign before real-world deployment.

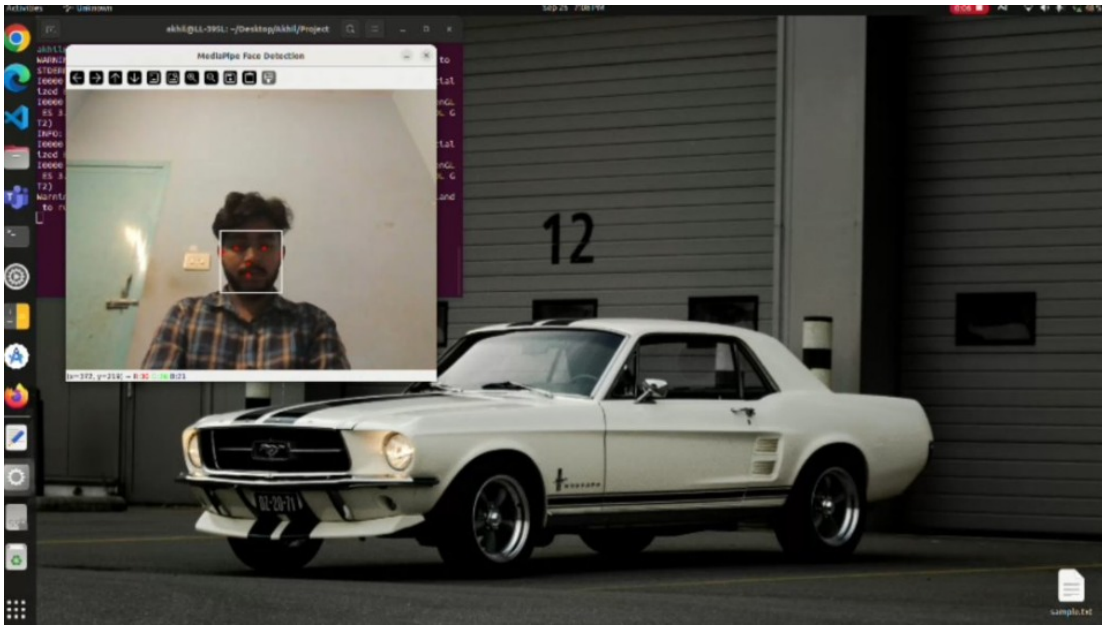

Figure 11.1: Person Detection

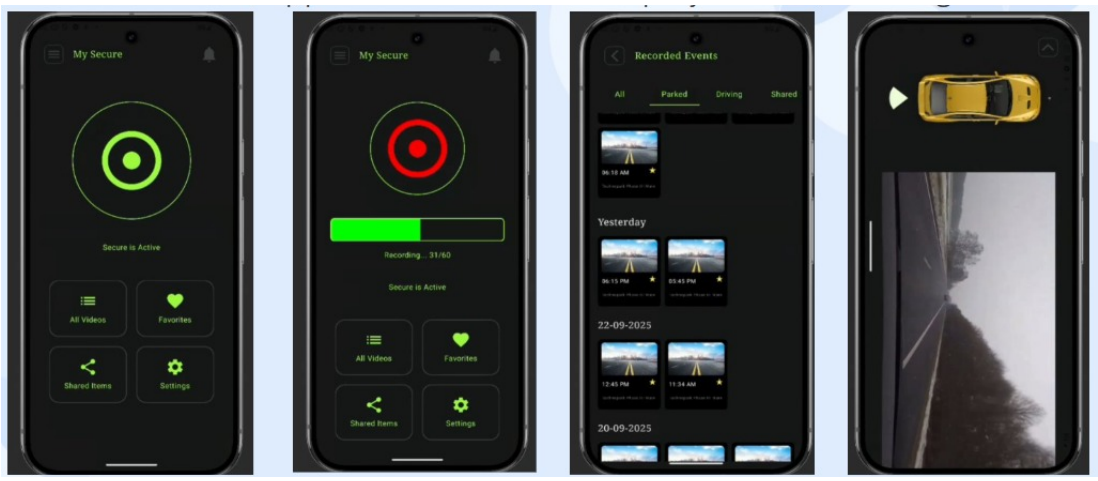

Figure 11.2: Mobile App

Chapter 12

Conclusion

This project successfully demonstrated a modular and event-driven vehicle surveillance system capable of detecting relevant events, recording buffered video clips, uploading them to a cloud backend, and notifying the user through a mobile application. By combining OpenCV- and MediaPipe-based event detection with GStreamer-based video processing and a lightweight web backend, the system moved beyond passive recording and provided a more responsive security workflow.

The use of independent processes connected through named pipes improved modularity and simplified system integration. The video manager preserved 30 seconds of pre-event footage and combined it with 30 seconds of post-event footage, thereby producing incident clips with meaningful context. The cloud manager and Android application further extended the usefulness of the system by making recorded events accessible remotely.

The work also highlighted practical limitations. Person detection accuracy can still be improved under challenging lighting and environmental conditions, and the system has not yet undergone large-scale quantitative benchmarking. In addition, further hardening is required in areas such as fault tolerance, network resilience, and secure deployment.

Future work may focus on improving detection robustness, measuring system latency under varied operating conditions, extending the trigger set to additional vehicle events, and migrating from a local backend to a production-ready secure cloud platform. Even in its current form, the prototype demonstrates the technical feasibility and practical value of intelligent, event-based vehicle surveillance.

References

[1] M. Patel, A. Shah, and R. Kumar, "Proposed method for car theft prevention techniques," *International Journal of Technology Research and Studies*, vol. 12, no. 3, pp. 45–59, 2024.

[2] A. Rocky, Q. J. Wu, and W. Zhang, "Review of accident detection methods using dashcam videos," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 8, pp. 8356–8367, 2024.

[3] M. T. Rehman, N. Khan, and S. Ali Shah, "Live view car surveillance system," *IEEE Access*, vol. 6, pp. 29488–29496, 2018.

[4] Tesla, Inc., "Sentry Mode: Guarding your Tesla," 2023. Available: `https://www.tesla.com/blog/sentry-mode-guarding-your-tesla`

[5] A. Arev, J. Park, A. Sheikh, *et al.*, "COSENS: Collaborative and opportunistic sensing of road events by vehicles' cameras," 2023.

[6] J. Park and S. Lee, "Cloud-assisted event detection in vehicular networks," 2022.

[7] I. Ahmad and M. A. Khan, "Real-time data streaming in connected vehicles," *Journal of Computer Networks and Communications*, 2022, pp. 1–11.

Appendix A

Annex: Software Requirements Definition (SRD)

A.1 Purpose

The Software Requirements Definition (SRD) describes the business context, stakeholder goals, and product boundary for the Intelligent Incident Surveillance Response System. The system is intended to provide event-based surveillance for vehicles by capturing relevant evidence, uploading it to a backend, and making it available through a mobile interface.

A.2 Scope

The product scope includes:

- Event generation from simulated and vision-based sources.
- Event orchestration and recording control.
- Buffered video recording with pre-event and post-event context.
- Upload of event clips and metadata to cloud/backend services.
- User notification and remote playback support through a mobile app.

The following are outside the current scope:

- Fleet-level analytics dashboards.
- Automated legal workflow for claims processing.
- Multi-vehicle live map tracking in production cloud.

A.3 Stakeholders

- Vehicle Owner: receives alerts and reviews incident evidence.
- Fleet Operator: monitors incident clips across assigned vehicles.
- Developer/Maintainer: configures trigger logic and backend services.
- System Administrator: maintains database, upload APIs, and storage.

A.4 Business and User Needs

- Reduce unnecessary continuous recording and storage overhead.
- Preserve contextual evidence before and after incidents.
- Improve response time with near-real-time mobile notifications.
- Provide simple remote review workflow for uploaded incidents.
- Keep architecture modular for easy maintenance and upgrades.

A.5 Assumptions and Constraints

- Camera feeds are available with acceptable quality for trigger processing.
- Local network and backend services are reachable during upload.
- Platform components run in a UNIX/Linux environment.
- Integration uses named pipes and API endpoints already defined in the project.
- Security hardening is baseline and can be extended in future revisions.

A.6 Success Criteria

The SRD is considered satisfied when the system can detect a supported event, generate and store a complete incident clip (with pre/post context), upload it with metadata, and make it discoverable from the mobile interface with user notification.

Appendix B

Annex: Software Requirements Specification (SRS)

B.1 Introduction

This SRS defines functional and non-functional requirements for the Intelligent Incident Surveillance Response System implementation documented in this project.

B.2 Functional Requirements

ID	Requirement	Description
FR-01	Event Detection	The system shall detect supported trigger events from simulation input and computer-vision input.
FR-02	Event Routing	The event manager shall validate events and dispatch recording commands to the recording subsystem.
FR-03	Pre-event Buffer	The video manager shall preserve at least 30 seconds of pre-event video context in memory.
FR-04	Post-event Capture	The system shall capture at least 30 seconds of post-event footage after a valid trigger.
FR-05	Clip Generation	The system shall generate a playable MP4 incident clip combining pre-event and post-event segments.

FR-06	Metadata Storage	The backend shall store clip metadata including timestamp, event type, and storage path.
FR-07	Cloud Upload	The cloud manager shall upload the generated clip to server storage over HTTP API.
FR-08	Notification	The mobile client shall receive event notifications after successful backend update.
FR-09	Incident Listing	The mobile client shall retrieve and display available incident recordings.
FR-10	Playback	The mobile client shall play selected recordings from backend-hosted media.

B.3 Non-Functional Requirements

- NFR-01 Performance: End-to-end flow from event to clip availability should complete within operationally acceptable delay for local deployment.
- NFR-02 Reliability: Module failures should be logged with enough detail for diagnosis and recovery.
- NFR-03 Maintainability: Components shall remain modular and independently testable.
- NFR-04 Compatibility: Generated clips shall be playable on common Android media stacks.
- NFR-05 Security: Upload and API access shall enforce authentication controls and secure transport where available.
- NFR-06 Scalability: Backend schema and API design shall support growth in incident volume.

B.4 External Interface Requirements

B.4.1 Software Interfaces

- Python modules for event processing and orchestration.
- Backend APIs implemented with Apache/PHP and MySQL.
- Firebase-based notification pathway for mobile alerting.

B.4.2 Communication Interfaces

- Named pipes for internal module signaling.
- HTTP endpoints for file upload and incident metadata operations.

B.5 Traceability Summary

- FR-01/FR-02 map to event modules.
- FR-03/FR-04/FR-05 map to recording service and encoding pipeline.
- FR-06/FR-07 map to backend upload and database handling.
- FR-08/FR-09/FR-10 map to mobile and notification integration.

Appendix C

Annex: Unit Testing (UT)

C.1 Objective

The unit-testing objective is to verify correctness of module-level logic before full integration. The focus is on deterministic event handling, recording boundaries, metadata consistency, and API response behavior.

C.2 Unit Test Scope

- Event parsing and classification logic.
- Trigger-to-record command dispatch.
- Buffer window assembly (pre-event and post-event).
- Upload request creation and retry behavior.
- Metadata generation and persistence mapping.

C.3 Test Environment

- Development host running the project Python modules.
- Local backend services (Apache, PHP, MySQL).
- Controlled test inputs (simulated honk/brake/person events).

C.4 Representative Unit Test Cases

Test ID	Target Unit	Input/Condition	Expected Result
UT-01	Event parser	Valid event payload	Event type resolved and routed without error
UT-02	Event parser	Invalid/malformed payload	Rejected safely; diagnostic log written
UT-03	Recording window logic	Trigger at time t	Clip contains buffer before t and capture after t
UT-04	Upload client	Backend reachable	HTTP success and metadata update acknowledged
UT-05	Upload client	Backend timeout/failure	Retry policy executed and failure status logged
UT-06	Metadata mapper	New incident clip data	Correct DB/API payload fields populated

C.5 Pass Criteria

- All critical unit tests pass for event routing and clip generation.
- Failure paths return controlled errors and emit actionable logs.
- Metadata fields used by the mobile application are complete and valid.

C.6 Risk-Based Notes

- Timing-sensitive tests can show variance depending on host load.
- Camera source interruptions should be tested separately under fault-injection scenarios.
- Notification timing validation is verified primarily in integration testing.